



Arvore de Huffman

Árvore de Huffman

- Uma árvore de Huffman é uma árvore estritamente binária usada para compressão de arquivos, visando reduzir o espaço necessário para armazená-los em disco ou o tempo necessário para transmiti-los por um canal de comunicação.
- Uma árvore binária completa, chamada de árvore de Huffman é construída recursivamente a partir da junção dos dois símbolos de menor probabilidade, que são então somados em símbolos auxiliares e estes símbolos auxiliares recolocados no conjunto de símbolos. O processo termina quando todos os símbolos forem unidos em símbolos auxiliares, formando uma árvore binária. A árvore é então percorrida, atribuindo-se valores binários de 1 ou 0 para cada aresta, e os códigos são gerados a partir desse percurso.

Árvore de Huffman

- De fato, como veremos a seguir, uma árvore de Huffman é uma estrutura que representa um código de tamanho variável, que minimiza o número de bits necessários para representar um arquivo.
- Diferentemente do código ASCII, que é padrão, o código de Huffman varia em função do arquivo a ser comprimido (e, portanto, deve ser criado especificamente para cada arquivo).
- O código de Huffman é livre de prefixo, isto é, para quaisquer dois caracteres a e b no alfabeto S , o código de a não é um prefixo do código de b . Essa propriedade garante que a descompressão de um arquivo s pode ser feita sem ambiguidade.

Árvore de Huffman

- Dada uma cadeia de caracteres s , representando um arquivo a ser comprimido, uma árvore de Huffman correspondente pode ser construída do seguinte modo:
 - Para cada caractere ASCII c , obtenha a frequência $f(c)$ de c em s .
 - Seja S o conjunto de caracteres ASCII c , tais que $f(c) > 0$.
 - Para cada caractere c em S , crie uma árvore binária correspondente com uma folha contendo $f(c)$. O conjunto de árvores criadas nesse passo é uma floresta. O valor na raiz de cada árvore da floresta é o peso da árvore.
 - Enquanto houver mais que uma árvore na floresta: remova duas árvores de pesos mínimos, digamos A_e e A_d ; em seguida, crie uma árvore binária A cuja raiz guarde a soma dos pesos de A_e e A_d e cujos filhos sejam A_e e A_d ; finalmente, insira a árvore A na floresta.

Árvore de Huffman

- No fim da repetição, a árvore que sobra é uma árvore de Huffman para o arquivo representado por s .
- Para obter os códigos comprimidos, rotule as ligações à esquerda com 0 e as ligações à direita com 1.
- Depois disso, o código de Huffman para cada caractere c é a sequência de bits que rotula o caminho que vai da raiz da árvore até a folha associada ao caractere c .

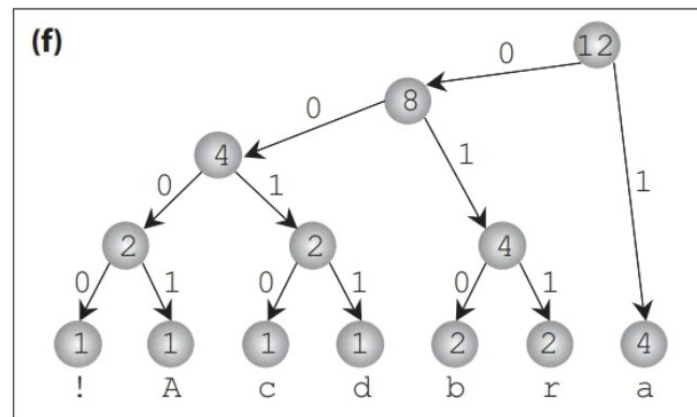
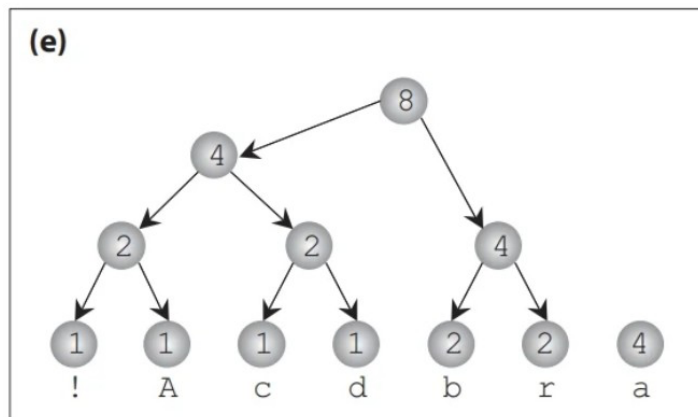
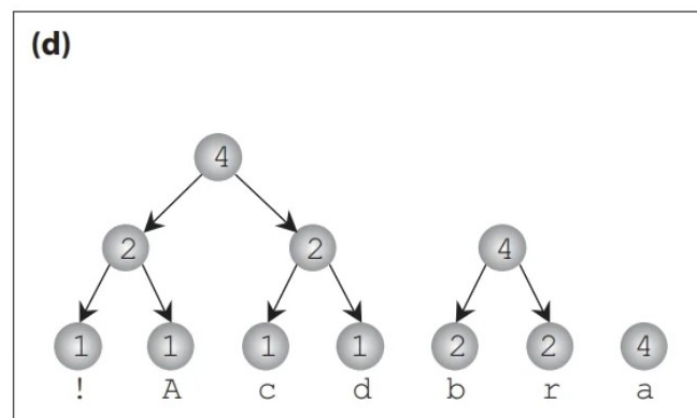
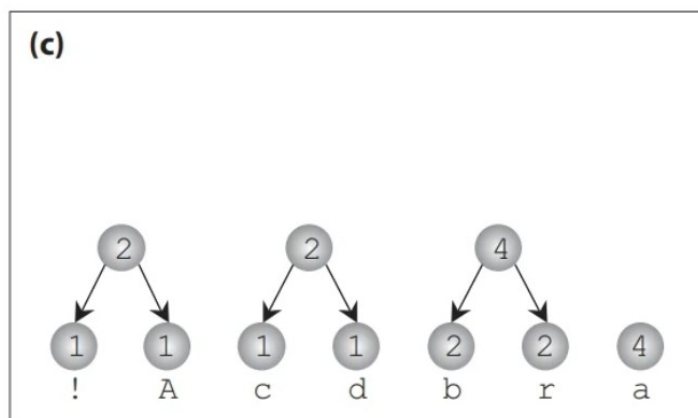
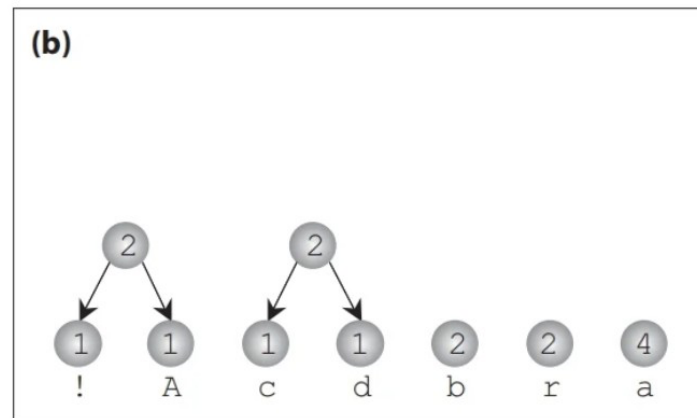
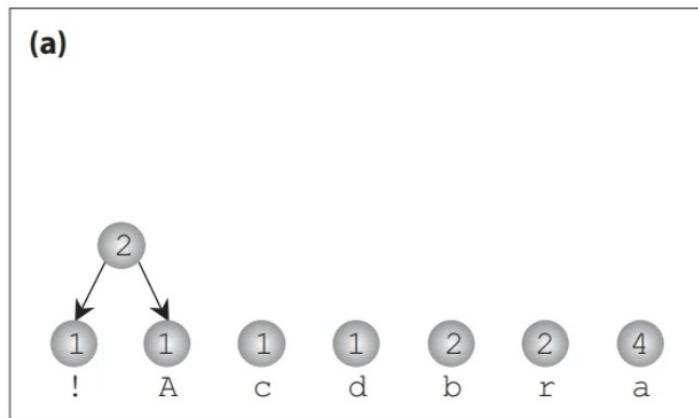


Figura 14.2 | Criação de uma árvore de Huffman para $\sigma = \text{"Abracadabra!"}$.

Código de Huffman – histograma de frequências

- Veremos uma implementação do código de Huffman. Dada uma cadeia de caracteres s , representando o conteúdo de um arquivo a ser comprimido, é criado um vetor de inteiros f tal que, para cada caractere ASCII c , $f[c]$ é a frequência de c em s :

```
int *freq(char *s) {  
    static int f[256];  
    for(int i=0; i<256; i++) f[i] = 0;  
    for(int i=0; s[i]; i++) f[s[i]]++;  
    return f;  
}
```

Código de Huffman – criação da árvore

- Para criar uma árvore de Huffman, vamos usar a seguinte estrutura de dados:

```
typedef struct arvh {  
    char chr;  
    int frq;  
    struct arvh *esq, *dir;  
} *Arvh;
```

Código de Huffman – criação da árvore

- Para criar uma árvore de Huffman, uma função recebe como entrada um caractere, sua frequência e duas árvores de Huffman e, como saída, ela devolve o endereço de um nó criado e preenchido com esses valores.
- Assim, cada folha numa árvore de Huffman guardará um caractere e sua frequência.

```
Arvh criaAH(Arvh e, char c, int f, Arvh d) {  
    Arvh n = malloc(sizeof(struct arvh));  
    n->esq = e;  
    n->chr = c;  
    n->frq = f;  
    n->dir = d;  
    return n;  
}
```

Código de Huffman – criação da árvore

- Uma floresta será representada por um vetor F com m árvores de Huffman, em ordem decrescente de peso. Para garantir a ordenação da floresta, usaremos a função abaixo. Dadas uma árvore A e uma floresta F com m árvores, essa função insere a árvore A em F, em ordem decrescente de peso, e incrementa m. O parâmetro m, que indica o tamanho da floresta, é passado por referência.

```
void insf(Arvh A, Arvh F[], int *m) {  
    int i = *m;  
    while( i>0 && F[i-1]->frq < A->frq ) {  
        F[i] = F[i-1];  
        i--;  
    }  
    F[i] = A;  
    (*m)++;  
}
```

Código de Huffman – criação da árvore

- Para remoção de uma árvore da floresta sabendo que F é um vetor de árvores em ordem decrescente de peso, basta remover a última árvore do vetor retornando ela como resposta:

```
Arvh remf(Arvh F[], int *m) {  
    if( *m == 0 ) return NULL;  
    return F[--(*m)];  
}
```

Código de Huffman – criação da árvore

- Para criar a árvore de Huffman a partir de uma cadeia de caracteres:

```
Arvh huffman(char *s) {  
    Arvh F[256];  
    int m=0;  
    int *f = freq(s);  
    for(int c=0; c<256; c++)  
        if( f[c]>0 ) insf(criaAH(NULL,c,f[c],NULL), F, &m);  
    while( m>1 ) {  
        Arvh d = remf(F, &m);  
        Arvh e = remf(F, &m);  
        insf(criaAH(e, '-', e->frq+d->frq, d), F, &m);  
    }  
    return F[0];  
}
```

Código de Huffman – codificando/comprimindo

- Construída a árvore A, agora podemos gerar o código binário a partir dela e da própria cadeia de caracteres. Para definir qual sequência binária para cada letra do alfabeto:

```
void tabela(Arvh A, char *H[]) {  
    static char c[256], t = -1;  
    if( A == NULL ) return;  
    if( A->esq == NULL && A->dir == NULL )  
        H[A->chr] = strndup(c, t+1);  
    else {  
        t++;  
        c[t] = '0'; tabela(A->esq, H);  
        c[t] = '1'; tabela(A->dir, H);  
        t--;  
    }  
}
```

Código de Huffman – codificando/comprimindo

- Sabendo o código de cada letra, podemos agora gerar o código a partir da string e da árvore de Huffman:

```
void comprimir(char *s, Arvh A) {  
    char *T[256];  
    for(int c=0; c<256; c++) T[c] = NULL;  
    tabela(A,T);  
    for(int i=0; s[i]; i++) printf("%s",T[s[i]]);  
    for(int c=0; c<256; c++) free(T[c]);  
}
```

Código de Huffman – decodificando/descomprimindo

- Para descomprimir uma cadeia de dígitos d, basta ter a árvore de Huffman A, que foi usada para comprimi-la. A cada valor binário, a árvore é percorrida até chegar no nos nós folha:

```
void descomprimir(char *d, Arvh A) {  
    if( A == NULL ) return;  
    Arvh n = A;  
    for(int i=0; d[i]; i++) {  
        n = (d[i]=='0') ? n->esq : n->dir;  
        if( n->esq == NULL && n->dir == NULL ) {  
            printf("%c",n->chr);  
            n = A;  
        }  
    }  
}
```

Código de Huffman – exemplo **Jararaca**

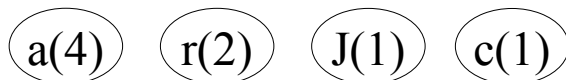
- Vamos ver como criar a árvore para Jararaca. Primeiro o histograma de frequência das letras de “Jararaca”:

4	2	1	1
a	r	J	c

Código de Huffman – exemplo **Jararaca**

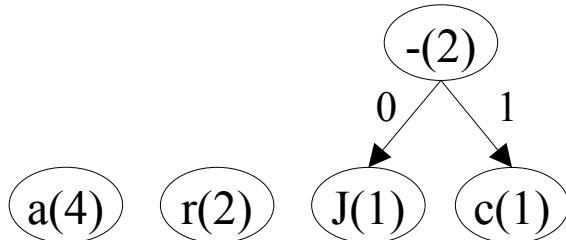
- Criando a árvore, primeiramente temos uma floresta com apenas o nó raiz/folha (um único nó por árvore):

4	2	1	1
a	r	J	c



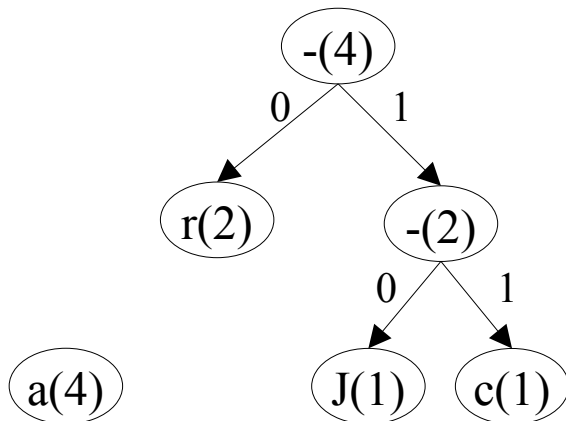
Código de Huffman – exemplo **Jararaca**

- A cada par com menores valores, juntamos os nós em uma nova árvore. Removemos “j” e “c” da floresta e criamos outra árvore com um nó raiz e ambos filhos dessa nova (folhas nesse caso):



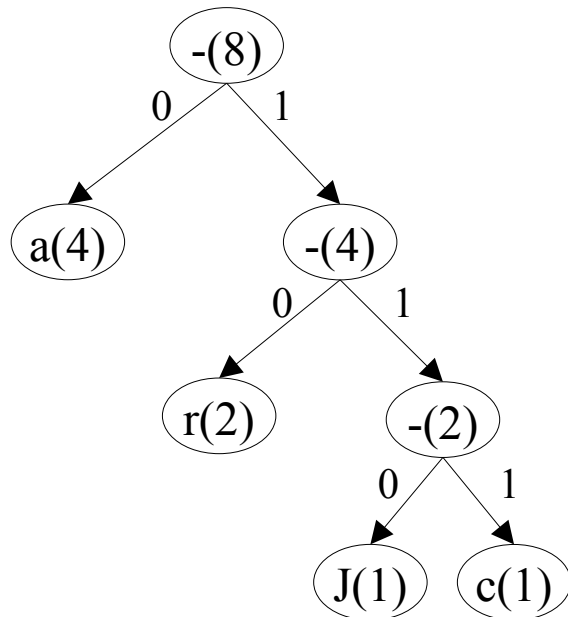
Código de Huffman – exemplo **Jararaca**

- A cada par com menores valores, juntamos os nós em uma nova árvore. Removemos “r” e última árvore e criamos outra com um nó raiz e ambos filhos dessa nova:



Código de Huffman – exemplo **Jararaca**

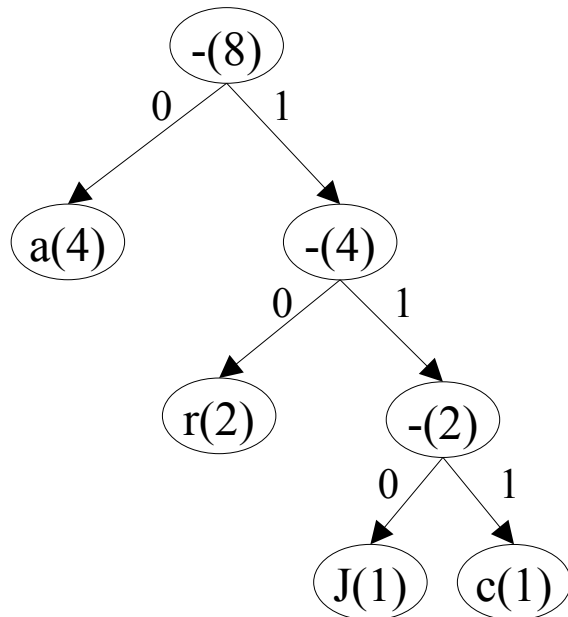
- A cada par com menores valores, juntamos os nós em uma nova árvore. Por fim, juntamos o “a” e última árvore e teremos a árvore de Huffman:



Código de Huffman – exemplo **Jararaca**

- Temos então o seguinte código para cada letra:

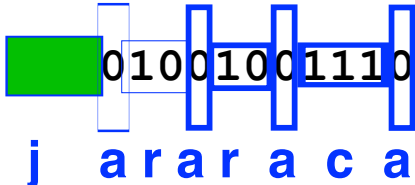
a: 0
r: 10
J: 110
c: 111

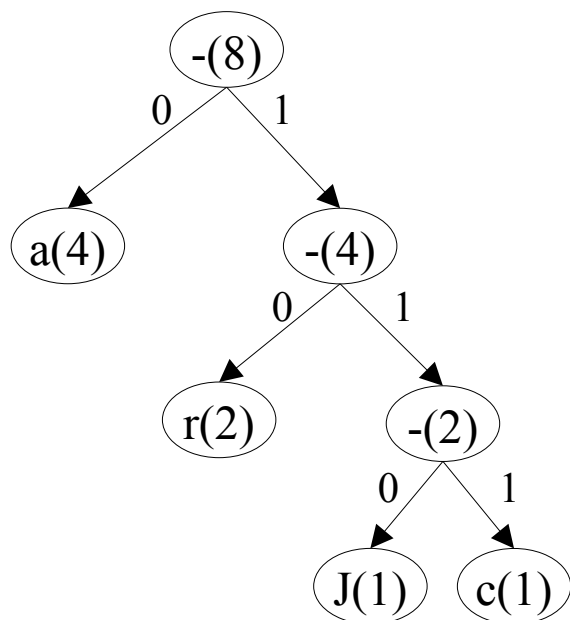


Código de Huffman – exemplo **Jararaca**

- Codificando:

a: 0
r: 10
J: 110
c: 111

código: 
j a r a r a c a





Arvore de Huffman